


**Instituto Tecnológico de Estudios Superiores de Zamora**

Maestría en Sistemas Computacionales

Asignatura: Ingeniería de Software

# Actividad 3. Investigación de artículos sobre la propuesta de tesis y la inteligencia artificial en Ingeniería de Software

Estudiante: Alberto García Rojas

Docente: Jorge Edgar Rojas Magdaleno

Fecha: 12 de noviembre de 2026

# 1. Artículos sobre la propuesta de tesis

## Artículo 1: SaaS Multi-tenancy

### Referencia bibliográfica

Bezemer, C.-P., & Zaidman, A. (2010). Multi-tenant SaaS applications: Maintenance dream or nightmare? En *Proceedings of the Joint ERCIM Workshop on Software Evolution (EVOL) and International Workshop on Principles of Software Evolution (IWPSE)* (pp. 88–92). ACM. https://doi.org/10.1145/1862372.1862393

### Enlace DOI

https://doi.org/10.1145/1862372.1862393

### Descripción

El software como servicio es un modelo de negocio que tiene múltiples clientes que pueden configurar con menos limitaciones la aplicación.

Multi-tenant se basa en una sola instancia con aislamiento en la base de datos para cada cliente y así evitar tener múltiples instancias, lo cual incrementa el costo de mantenimiento; la desventaja es que es más complejo de crear.

El propósito del artículo es analizar si atender a varios clientes con una sola instancia facilita o complica el mantenimiento y la evolución del software. Los beneficios que ofrece esta arquitectura son:

- Mayor utilización de recursos de hardware
- Mayor facilidad y menor costo de mantenimiento
- Reducción de costos
- Oportunidades de agregación y análisis de datos de todos los clientes

### Palabras clave

Software como servicio, multi-tenancy, mantenimiento de software, evolución del software, personalización.

### Justificación

Optibyte está pensado como un SaaS multiempresa y el artículo advierte sobre los riesgos y dificultades del diseño del modelo de datos y las limitaciones del proyecto, sobre todo cuando cada cliente pide personalizaciones.

# Artículo 2: Seguridad y Privacidad

## Referencia bibliográfica

Fernández-Alemán, J. L., Señor, I. C., Lozoya, P. Á. O., & Toval, A. (2013). Security and privacy in electronic health records: A systematic literature review. *Journal of Biomedical Informatics, 46*(3), 541–562. https://doi.org/10.1016/j.jbi.2012.12.003

## Enlace DOI

https://doi.org/10.1016/j.jbi.2012.12.003

## Descripción

Es una revisión sistemática de 49 estudios sobre la seguridad y privacidad de los expedientes clínicos electrónicos. Su propósito es identificar qué medidas se proponen para proteger estos sistemas.

En el artículo se habla del control de acceso a los datos y el modelo preferido es el *modelo de roles*; también se recomienda una bitácora de cambios y autenticación reforzada. Además, señala que pocos trabajos abordan la capacitación del personal en temas de seguridad.

## Palabras clave

Expediente clínico electrónico, seguridad, privacidad, control de acceso, bitácora de auditoría.

## Justificación

Optibyte es un software dedicado al sector salud y guarda información médica de los pacientes. Actualmente, los datos se administran en hojas de cálculo, sin ningún control de acceso a ellos. El software que se construye pretende resolver eso, además de crear la bitácora de cambios y proveer un control de acceso mediante roles y autenticación.

# Artículo 3: Scrum en Equipos Pequeños

## Referencia bibliográfica

Senabre Hidalgo, E. (2019). Adapting the scrum framework for agile project management in science: Case study of a distributed research initiative. *Heliyon, 5*(3), Artículo e01447. https://doi.org/10.1016/j.heliyon.2019.e01447

## Enlace DOI

https://doi.org/10.1016/j.heliyon.2019.e01447

## Descripción

El artículo es un estudio de caso sobre cómo adaptar el marco de trabajo Scrum en un equipo pequeño y distribuido. Su propósito es saber si los principios y herramientas de Scrum ayudan a coordinar las tareas de un equipo fuera del entorno de una empresa de software.

El equipo trabajó con Scrum durante 10 semanas y después se recogió su opinión. Los resultados muestran que los integrantes percibieron una mejor coordinación y trabajo en equipo, aunque no todos estuvieron de acuerdo en la utilidad de algunas herramientas. El autor concluye que Scrum se puede adaptar, pero hay que ajustar sus prácticas al tamaño y las condiciones del equipo.

## Palabras clave

Scrum, metodologías ágiles, gestión de proyectos, estudio de caso, equipos pequeños.

## Justificación

Optibyte se desarrolla con un marco ágil basado en Scrum con iteraciones de dos semanas y con un equipo muy pequeño (un solo integrante). Este artículo sustenta la elección de la metodología y muestra cómo adaptar Scrum cuando no se cuenta con un equipo completo, que es una de las limitaciones del proyecto, ya que será creado por un solo desarrollador.

# 2. Artículos sobre IA en Ingeniería de Software

## Artículo 4: LLM en Ingeniería de Software

### Referencia bibliográfica

Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., & Wang, H. (2024). Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology, 33*(8), Artículo 220. https://doi.org/10.1145/3695988

### Enlace DOI

https://doi.org/10.1145/3695988

### Descripción

Es una revisión sistemática sobre el uso de los modelos de lenguaje grandes (LLM) en la ingeniería de software. Su propósito es mostrar qué modelos se usan, con qué datos y técnicas, y en qué tareas del ciclo de vida del software.

Revisa cientos de trabajos publicados entre 2017 y 2024 y los clasifica por tarea: requisitos, diseño, generación de código, pruebas y mantenimiento, entre otras. También señala los retos pendientes y las líneas de investigación futuras.

### Palabras clave

Modelos de lenguaje grandes, ingeniería de software, revisión sistemática, generación de código, inteligencia artificial.

### Justificación

Sirve como mapa general del tema: permite ubicar en qué fases de la ingeniería de software ya se aplica la IA y cuáles siguen poco exploradas. Además, el uso de LLM ayudará a agilizar el desarrollo del proyecto ya que solo cuenta con un solo desarrollador.

# Artículo 5: Generación de Código con IA

## Referencia bibliográfica

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. de O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., … Zaremba, W. (2021). *Evaluating large language models trained on code* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2107.03374

## Enlace DOI

https://doi.org/10.48550/arXiv.2107.03374

## Descripción

El artículo presenta Codex, un modelo de lenguaje entrenado con código fuente, y tiene como propósito medir qué tan bien resuelve problemas de programación.

Introduce el banco de pruebas HumanEval, con 164 problemas que se verifican con pruebas unitarias. Muestra que generar varias soluciones por problema aumenta la tasa de acierto y discute las limitaciones y los riesgos de seguridad de este tipo de herramientas.

## Palabras clave

Codex, generación de código, HumanEval, modelos de lenguaje, evaluación.

## Justificación

Es el trabajo que dio origen a asistentes como GitHub Copilot. Ayuda a entender cómo se evalúa el código generado por IA y por qué siempre hay que revisarlo y probarlo antes de usarlo. La programación agentica bien planeada y supervisada será fundamental para poder desarrollar por completo un software de gran tamaño.

# Artículo 6: Productividad con Asistentes de IA

## Referencia bibliográfica

Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). *The impact of AI on developer productivity: Evidence from GitHub Copilot* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2302.06590

## Enlace DOI

https://doi.org/10.48550/arXiv.2302.06590

## Descripción

El artículo estudia el efecto de un asistente de programación con IA en la productividad de los desarrolladores. Su propósito es medir, con un experimento controlado aleatorizado, si GitHub Copilot reduce el tiempo de desarrollo.

Los desarrolladores que usaron Copilot terminaron la tarea (un servidor HTTP en JavaScript) 55.8 % más rápido que el grupo de control, y el beneficio fue mayor en las personas con menos experiencia.

## Palabras clave

GitHub Copilot, productividad, experimento controlado, asistente de programación, inteligencia artificial.

## Justificación

Aporta evidencia cuantitativa del uso de la IA en la fase de programación y un ejemplo de diseño experimental. También permite dimensionar el beneficio que podría tener usar asistentes de IA en el desarrollo de Optibyte.

# Anexos

**Anexo 1.** Bezemer y Zaidman (2010):
https://dl.acm.org/doi/epdf/10.1145/1862372.1862393

**Anexo 2.** Fernández-Alemán et al. (2013)

https://www.sciencedirect.com/science/article/pii/S1532046412001864?via%3Dihub

**Anexo 3.** Senabre Hidalgo (2019)

https://pmc.ncbi.nlm.nih.gov/articles/PMC6441834/

**Anexo 4.** Hou et al. (2024)

https://dl.acm.org/doi/10.1145/3695988

**Anexo 5.** Chen et al. (2021)

https://arxiv.org/pdf/2107.03374

**Anexo 6.** Peng et al. (2023)

https://arxiv.org/pdf/2302.06590